Jake DeMuesy

CS 499

6/29/26

Category One – Narrative

Narratives: Provide a narrative to accompany each artifact in your ePortfolio. The narrative that you write for each artifact should explain why you included the artifact in your ePortfolio and reflect on the process you used to create the artifact. The narrative should focus less on the actual creation of the artifact and more on the learning that happened through the creation of the artifact. Address the following items for each artifact:

Briefly **describe** the artifact. What is it? When was it created?

This artifact is a C++ OpenGL application that renders an interactive 3D desk scene -- a desktop with a globe, coffee mug, book stacks, a desk lamp, a pencil holder, and some scattered clutter. The original version was built during CS 330, Computational Graphics and Visualization, around late 2023. At that point it was mostly a big flat source file doing everything in one place -- loading geometry, setting up textures, configuring lights, and rendering, all kind of tangled together. The enhanced version for the capstone refactored that into a proper class structure with separate responsibilities for scene management, scene graph traversal, shader coordination, and lighting configuration.

Justify the inclusion of the artifact in your ePortfolio. Why did you select this item? What specific components of the artifact showcase your skills and abilities in software development? How did the enhancement improve the artifact? What specific skills did you demonstrate in the enhancement?

I picked this project because C++ is genuinely harder to write cleanly than a lot of languages (for me), and the refactor gave me a chance to demonstrate that I can apply real software engineering patterns in a lower-level environment. The original code worked fine as a CS 330 submission but it wasn't something I'd want a hiring manager to read through. The enhanced version is.

The specific components that show the most are the SceneGraph composite pattern -- SceneNode and RenderableObject with proper virtual dispatch -- the ShaderProgram abstraction with its overloaded SetUniform() methods, and the LightingConfig file parser that reads point light definitions from an external text file at runtime rather than hardcoding them. Those three things together take the artifact from "student assignment" to something closer to a real engine component.

The most recent round of changes pushed it a step further. Pulling texture management out of SceneManager into its own TextureManager class is a direct application of the Single Responsibility Principle -- SceneManager was doing too many things. The TEXTURE_INFO struct was also public inside SceneManager even though nothing outside the class ever touched it, so moving it private inside TextureManager as TextureRecord cleaned up that encapsulation issue. And updating the comments throughout -- especially in SceneGraph.cpp and LightingConfig.cpp -- to explain the engineering reasoning behind decisions rather than just describing what the code does makes the codebase actually readable as a portfolio piece.

Reflect on the process of enhancing the artifact. What did you learn as you were creating it and improving it? What challenges did you face? How did you incorporate feedback as you made changes to the artifact? How was the artifact improved? Which course outcomes did you partially or fully meet with your enhancements? Which do you feel were not met?

The biggest thing I learned through this process is that "works correctly" and "well-designed" are pretty different bars. The CS 330 submission cleared the first bar without much trouble. Getting to the second one took a lot more thought about where responsibilities actually belong.

The scene graph refactor was the core challenge early on. The composite pattern -- where SceneNode can hold child nodes and RenderableObject overrides RenderSelf() -- isn't complicated in theory, but getting the transform accumulation right so parent matrices correctly propagate down the tree took quite a lot debugging. The RenderContext struct as a dependency injection vehicle for the shader program, mesh collection, and texture resolver was a design decision I had to think through carefully, because the alternative was passing five separate parameters through every recursive call, which gets messy fast.

The feedback from Dr. Bolton after the first submission was pretty pointed on two things: the SRP issue and the proper comments. On SRP, I'll admit my first instinct was to dismiss it a bit -- SceneManager felt fine to me because the texture-related methods were at least grouped together. But once I actually extracted TextureManager, the class boundaries made a lot more sense and SceneManager.h got noticeably cleaner. That's the kind of thing that's easy to rationalize away until you just do it and see the result. On comments, the feedback was that comments should explain *why* a decision was made, not just describe what the line does. That one stuck with me. Going back through SceneGraph.cpp and LightingConfig.cpp and rewriting comments to explain things like why linear scan is acceptable for the texture slot lookup, or why the TRS matrix order is the way it is, forced me to actually articulate the reasoning I'd been carrying around implicitly. That was a very a useful exercise!

The course outcomes this enhancement covers most directly are the design and engineering one -- using established patterns like composite and SRP to build something maintainable -- and the professional communication outcome, since the narrative and the code comments both feed into that. The security outcome is only touched lightly here, mainly through access modifier discipline (private members, const references) rather than anything authentication-related -- that was handled more fully in Enhancement Three with the dashboard RBAC work. The algorithms outcome was addressed more directly in Enhancement Two with the pirate agent, so I'd say Enhancement One is strongest on design principles and documentation.